

Probability & Random Process Algorithms

A complete placement + competitive programming guide covering probability theory, random variables, expectation, Markov chains, and random processes.

BASIC PROBABILITY

1. Overview

Probability is the measure of the likelihood of an event occurring. It ranges from 0 (impossible) to 1 (certain). In programming contexts, probability helps us analyze randomized algorithms, model uncertainty, and solve problems involving random events.

2. Intuition

Simple Explanation

If you flip a fair coin, there are 2 possible outcomes: heads or tails. The probability of heads is $1/2 = 0.5$. In general:

$$P(\text{event}) = \frac{\text{number of favorable outcomes}}{\text{total number of possible outcomes}}$$

Analogy: The Dartboard

Imagine throwing a dart at a board divided into equal sections. The chance of hitting any particular section is proportional to its area. If one section takes up $1/4$ of the board, you have a 25% chance of hitting it. Probability is just a way of quantifying "how much of the total space" an event occupies.

Step-by-Step Reasoning

1. Define the sample space (all possible outcomes).
2. Define the event (subset of outcomes you care about).
3. Count or measure the favorable outcomes.
4. Divide by total outcomes — but only if all outcomes are equally likely.

Why It Works

Probability is built on three axioms:

- **Non-negativity:** $P(E) \geq 0$ for any event E .
- **Normalization:** $P(\text{Sample Space}) = 1$.
- **Additivity:** For mutually exclusive events, $P(A \cup B) = P(A) + P(B)$.

These axioms let us build consistent models of randomness.

3. When to Use It

- Analyzing randomized algorithms (quickselect, randomized quicksort)
 - Solving counting problems where outcomes are equally likely
 - Modeling games of chance, dice, cards, coins
 - Computing failure rates or reliability of systems
 - Any problem that says "randomly", "uniformly", "probability", "chance", "expected"
-

4. When Not to Use It

- When outcomes are not equally likely and you can't compute weights
 - When the sample space is infinite and you need measure theory
 - When deterministic counting (combinatorics alone) suffices
 - When the problem involves conditional dependencies that are better handled with Bayes
-

5. Core Concepts

Sample Space (S)

The set of all possible outcomes. For a die roll, $S = \{1, 2, 3, 4, 5, 6\}$.

Event (E)

A subset of the sample space. $E = \text{"rolling an even number"} = \{2, 4, 6\}$.

Probability of an Event

$P(E) = |E| / |S|$ for equally likely outcomes.

Mutually Exclusive Events

Events that cannot happen simultaneously. $P(A \cap B) = 0$.

Independent Events

Events where the occurrence of one does not affect the other. $P(A \cap B) = P(A) \times P(B)$.

Complement

$P(\text{not } E) = 1 - P(E)$.

6. Step-by-Step Algorithm

To compute probability of an event under uniform distribution:

1. Identify the sample space S .
 2. Count total outcomes $|S|$.
 3. Identify the event E .
 4. Count favorable outcomes $|E|$.
 5. Compute $P(E) = |E| / |S|$.
 6. Simplify fraction or convert to decimal.
-

7. Dry Run

Problem: What is the probability of rolling a sum of 7 with two fair dice?

Step 1: Sample space = $6 \times 6 = 36$ possible ordered pairs.

Step 2: Favorable outcomes (sum = 7): (1,6), (2,5), (3,4), (4,3), (5,2), (6,1) → 6 outcomes.

Step 3: $P(\text{sum} = 7) = 6/36 = 1/6 \approx 0.1667$.

Die 1	Die 2	Sum
1	6	7

Die 1	Die 2	Sum
2	5	7
3	4	7
4	3	7
5	2	7
6	1	7

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Compute probability of event with |E| favorable outcomes out of |S| total
double basicProbability(int favorable, int total) {
    if (total == 0) return 0.0;
    return (double)favorable / total;
}

// Probability of independent events ALL occurring
double allIndependent(const vector<double>& probs) {
    double ans = 1.0;
    for (double p : probs) ans *= p;
    return ans;
}

// Probability of at least one of independent events occurring
double atLeastOne(const vector<double>& probs) {
    double none = 1.0;
    for (double p : probs) none *= (1.0 - p);
    return 1.0 - none;
}

int main() {
    // Sum of two dice = 7
    cout << "P(sum=7 with two dice): " << basicProbability(6, 36) << "\n";

    // Probability of heads 3 times in a row
    cout << "P(3 heads): " << allIndependent({0.5, 0.5, 0.5}) << "\n";

    // At least one head in 3 flips
    cout << "P(at least one head): " << atLeastOne({0.5, 0.5, 0.5}) << "\n";

    return 0;
}
```

9. Python Implementation

```
def basic_probability(favorable: int, total: int) -> float:
    if total == 0:
        return 0.0
    return favorable / total
```

```
def all_independent(probs: list) -> float:
    ans = 1.0
    for p in probs:
        ans *= p
    return ans

def at_least_one(probs: list) -> float:
    none = 1.0
    for p in probs:
        none *= (1.0 - p)
    return 1.0 - none

# Sum of two dice = 7
print(f"P(sum=7 with two dice): {basic_probability(6, 36)}")

# Probability of heads 3 times in a row
print(f"P(3 heads): {all_independent([0.5, 0.5, 0.5])}")

# At least one head in 3 flips
print(f"P(at least one head): {at_least_one([0.5, 0.5, 0.5])}")
```

10. Code Explanation

- **basicProbability**: Straight division of favorable by total. Handles zero total guard.
- **allIndependent**: Multiplies probabilities because independent events require all to occur.
- **atLeastOne**: Uses complement. Compute probability of none occurring, subtract from 1.
- All functions return **double** for precision. In CP, be mindful of floating point errors.

11. Complexity Analysis

Operation	Time	Space
Single probability	O(1)	O(1)
All independent (n events)	O(n)	O(1)
At least one (n events)	O(n)	O(1)

12. Common Patterns

Pattern	Identification	Approach	Example
Basic counting	"randomly chosen", "uniformly"	Count favorable / total	P(even on die)
At least one	"at least one", "probability of success"	$1 - P(\text{all fail})$	P(at least one 6 in 4 rolls)
Independent events	"and", "both", "all"	Multiply probabilities	P(heads AND heads)
Mutually exclusive	"either", "or" (no overlap)	Add probabilities	P(roll 2 or 4)

13. Common Mistakes

- Assuming events are independent when they are dependent.
- Forgetting that "or" in probability includes both (union, not exclusive or).
- Dividing by total outcomes when outcomes are not equally likely.
- Integer division: **favorable** / **total** in C++ with ints gives 0.

- Not reducing fractions or using `double` when precision matters.

14. Edge Cases

- Total outcomes = 0 (undefined).
- Probability = 0 (impossible event).
- Probability = 1 (certain event).
- Very large counts causing overflow (use `long long` or `double`).
- Floating point precision for very small probabilities.

15. Variations

Variation	What Changes	When Used	Importance
Geometric probability	Continuous sample space	Random points on a line/area	Moderate
Empirical probability	Based on observed frequency	ML, statistics	Low for CP
Subjective probability	Based on belief	Bayesian inference	Low for CP

16. Related Concepts

- **Combinatorics:** Counting favorable/total outcomes uses permutations and combinations.
- **Expected Value:** Averages over probability distributions.
- **Conditional Probability:** Probability given prior knowledge.
- **Bayes Theorem:** Reversing conditional probabilities.

17. Practice Problems

Easy

- **Dice Roll** (LeetCode) — Basic probability calculation.
- **Probability of a Two Boxes Having Same Number of Distinct Balls** (LeetCode) — Counting + probability.

Medium

- **New 21 Game** (LeetCode 837) — Dynamic programming with probability.
- **Toss Strange Coins** (LeetCode) — DP with probability.
- **Random Pick with Weight** (LeetCode 528) — Weighted random selection.

Hard

- **Probability of a Knight's Tour** — Combinatorial probability.
- **Random Point in Non-overlapping Rectangles** (LeetCode 497) — Weighted probability.

18. Interview Explanation

"Probability is a measure of how likely an event is, ranging from 0 to 1. For equally likely outcomes, it's the count of favorable outcomes divided by total outcomes. The core rules are: the complement rule ($P(\text{not } A) = 1 - P(A)$), the addition rule for mutually exclusive events, and the multiplication rule for independent events. In coding, probability is used to analyze randomized algorithms, design sampling strategies, and solve problems involving random selection."

19. Revision Notes

- $P(E) = \text{favorable} / \text{total (equally likely outcomes)}$
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- $P(A \cap B) = P(A) \times P(B)$ if independent
- $P(\text{not } A) = 1 - P(A)$
- Always check if outcomes are equally likely
- Use **double** for fractional results
- Complement trick: "at least one" = $1 - \text{"none"}$

20. Final Cheat Sheet

Concept	Formula	Use Case
Basic probability	favorable / total	Uniform random selection
Complement	$1 - P(E)$	"At least one" problems
Union (non-exclusive)	$P(A) + P(B) - P(A \cap B)$	"Either A or B"
Intersection (independent)	$P(A) \times P(B)$	"Both A and B"
Intersection (dependent)	$P(A) \times P(B A)$	Sequential events

EXPECTED VALUE

1. Overview

Expected value (expectation) is the long-run average value of a random variable. If you repeat an experiment many times, the average of the outcomes converges to the expected value. In CP, expected value is used to compute average outcomes of randomized processes.

2. Intuition

Simple Explanation

If you roll a fair die, the expected value is $(1+2+3+4+5+6)/6 = 3.5$. You never actually roll a 3.5, but over many rolls, the average approaches 3.5.

Analogy: The Carnival Game

A game costs \$5 to play. You win \$10 with probability 0.3, and \$0 with probability 0.7. Your expected winnings = $10 \times 0.3 + 0 \times 0.7 = \3 . On average, you lose \$2 per game. The expected value tells you whether the game is fair.

Step-by-Step Reasoning

1. For each possible outcome, multiply its value by its probability.
2. Sum all these products.
3. The result is the expected value.

Why It Works

Expected value is a weighted average where weights are probabilities. By the law of large numbers, as the number of trials increases, the sample mean converges to the expected value.

3. When to Use It

- Computing average runtime of randomized algorithms
- Problems asking "expected number of steps/rolls/trials"
- Game theory problems asking "expected winnings"
- Any problem with "expected value", "expectation", "average outcome"
- DP problems where state transitions have probabilities

4. When Not to Use It

- When you need the actual distribution, not just the average
- When the expected value is not informative (e.g., bimodal distributions)
- When the random variable has infinite expectation (Cauchy distribution)
- When you need median or mode instead of mean

5. Core Concepts

Definition

For a discrete random variable X :

$$E[X] = \sum_x x \cdot P(X = x)$$

Continuous Case

$$E[X] = \int_{-\infty}^{\infty} x \cdot f(x) dx$$

Expectation of a Function

$$E[g(X)] = \sum_x g(x) \cdot P(X = x)$$

Properties

- **Linearity:** $E[aX + bY] = aE[X] + bE[Y]$ (always, no independence needed)
- **Constant:** $E[c] = c$
- **Non-negativity:** If $X \geq 0$, then $E[X] \geq 0$

6. Step-by-Step Algorithm

To compute expected value of a discrete random variable:

1. List all possible outcomes of the random variable.
2. Determine the probability of each outcome.
3. Multiply each outcome value by its probability.
4. Sum all products.

7. Dry Run

Problem: A fair coin is flipped. You win \$2 for heads, lose \$1 for tails. Find expected winnings.

Outcome	Value (x)	Probability (p)	$x \times p$
Heads	+2	0.5	1.0
Tails	-1	0.5	-0.5

Outcome	Value (x)	Probability (p)	$x \times p$
Total			0.5

$$E[X] = 2 \times 0.5 + (-1) \times 0.5 = 1 - 0.5 = \$0.50$$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Expected value from parallel arrays of outcomes and probabilities
double expectedValue(const vector<double>& values, const vector<double>& probs) {
    assert(values.size() == probs.size());
    double ev = 0.0;
    for (size_t i = 0; i < values.size(); i++) {
        ev += values[i] * probs[i];
    }
    return ev;
}

// Expected value of a die roll
double dieRollEV() {
    vector<double> values = {1, 2, 3, 4, 5, 6};
    vector<double> probs = {1.0/6, 1.0/6, 1.0/6, 1.0/6, 1.0/6, 1.0/6};
    return expectedValue(values, probs);
}

// Expected number of trials to get first success (Geometric distribution)
double geometricEV(double successProb) {
    return 1.0 / successProb;
}

int main() {
    cout << "Die roll EV: " << dieRollEV() << "\n"; // 3.5
    cout << "Coin flip game EV: " << expectedValue({2, -1}, {0.5, 0.5}) << "\n"; // 0.5
    cout << "Geometric EV (p=0.3): " << geometricEV(0.3) << "\n"; // 3.33...
    return 0;
}
```

9. Python Implementation

```
def expected_value(values: list, probs: list) -> float:
    assert len(values) == len(probs)
    return sum(v * p for v, p in zip(values, probs))

def die_roll_ev() -> float:
    values = [1, 2, 3, 4, 5, 6]
    probs = [1/6] * 6
    return expected_value(values, probs)

def geometric_ev(success_prob: float) -> float:
    return 1.0 / success_prob

print(f"Die roll EV: {die_roll_ev()}")
```

```
print(f"Coin flip game EV: {expected_value([2, -1], [0.5, 0.5])}")
print(f"Geometric EV (p=0.3): {geometric_ev(0.3)}")
```

10. Code Explanation

- `expectedValue`: Straightforward weighted sum. $O(n)$ time.
- `dieRollEV`: Uniform distribution over 6 values.
- `geometricEV`: For repeated independent trials with success probability p , expected trials until first success = $1/p$.
- Assertion ensures arrays are same length. In CP, you may skip this.

11. Complexity Analysis

Operation	Time	Space
Basic EV calculation (n outcomes)	$O(n)$	$O(n)$
Geometric EV	$O(1)$	$O(1)$
EV via DP (state space S)	$O(S \times \text{transitions})$	$O(S)$

12. Common Patterns

Pattern	Identification	Approach	Example
Geometric distribution	"expected number of trials until success"	$E = 1/p$	Expected coin flips for heads
DP with expectation	"expected value after n steps"	$DP[i] = \text{sum of } P(\text{next}) \times (1 + DP[\text{next}])$	Expected dice rolls to reach sum
Indicator variables	"expected count of something"	Use linearity of expectation	Expected number of fixed points
EV of a game	"expected winnings"	Weighted sum of outcomes	Casino game analysis

13. Common Mistakes

- Forgetting to multiply by probability (just averaging values).
- Assuming $E[f(X)] = f(E[X])$ — Jensen's inequality says otherwise.
- Using integer division in C++.
- Not normalizing probabilities to sum to 1.
- Confusing expected value with most likely value.

14. Edge Cases

- Zero-probability outcomes (can be ignored).
- Infinite expected value (e.g., St. Petersburg paradox).
- Negative values (expected value can be negative).
- Very large values $\times$ very small probabilities (floating point underflow).

15. Variations

Variation	What Changes	When Used	Importance
Conditional expectation	Expectation given an event	Bayesian analysis	High
Law of total expectation	$E[X] = E[E[X Y]]$	Breaking down complex expectations	High for CP
Expectation of max/min	$E[\max(X,Y)]$	Order statistics, auctions	Moderate

16. Related Concepts

- **Linearity of Expectation:** The most powerful tool for computing complex expectations.
- **Law of Large Numbers:** Why expected value matters in practice.
- **Variance:** Measures spread around the expected value.
- **Markov Chains:** Expected hitting times in random processes.

17. Practice Problems

Easy

- **Expected Number of Tosses to Get N Consecutive Heads** (GFG) — Geometric distribution.
- **Dice Rolls** (CSES 1095) — Basic EV.

Medium

- **Random Flips** (LeetCode) — DP with expectation.
- **Expected Value of a Game** (Codeforces) — Game theory + EV.
- **Sereja and Dima** (Codeforces) — Game with expectation.

Hard

- **Expected Number of Trials to Get All Faces of a Die** (Coupon Collector) — Coupon collector variation.
- **Random Walk Expected Hitting Time** — Markov chain analysis.

18. Interview Explanation

"Expected value is the long-run average of a random variable. For discrete variables, it's the sum of each outcome multiplied by its probability. The key property is linearity — $E[X+Y] = E[X] + E[Y]$ even when X and Y are dependent. This makes it extremely useful for breaking down complex random processes into simpler pieces. In DP problems, expected value often satisfies recurrence relations where the expected value at a state is 1 plus the weighted average of expected values of next states."

19. Revision Notes

- $E[X] = \sum x \times P(X = x)$
- $E[aX + b] = aE[X] + b$
- Linearity: $E[X + Y] = E[X] + E[Y]$ (always)
- Geometric: $E = 1/p$ for first success
- DP for EV: $E[\text{state}] = 1 + \sum P(\text{next}) \times E[\text{next}]$
- Common trap: $E[f(X)] \neq f(E[X])$

20. Final Cheat Sheet

Formula	Use
$E[X] = \sum x \cdot P(x)$	Basic discrete EV

Formula	Use
$E[aX+b] = aE[X]+b$	Scaling
$E[X+Y] = E[X]+E[Y]$	Always true
Geometric: $E = 1/p$	First success
DP: $E[i] = 1 + \sum p_j \cdot E[j]$	State-based EV

LINEARITY OF EXPECTATION

1. Overview

Linearity of expectation states that the expected value of a sum of random variables equals the sum of their individual expected values, regardless of whether the variables are independent. This is one of the most powerful tools in probability for CP and placement problems.

2. Intuition

Simple Explanation

If you have two random variables X and Y , then:

$$E[X + Y] = E[X] + E[Y]$$

This holds even if X and Y are completely dependent on each other. No independence condition is needed.

Analogy: The Shopping Cart

You go shopping and buy apples (random cost) and bananas (random cost). The expected total bill is simply the expected cost of apples plus the expected cost of bananas — even if apple prices influence banana prices. The average of the sum is always the sum of the averages.

Step-by-Step Reasoning

1. Write the random variable of interest as a sum of simpler indicator variables.
2. Compute the expected value of each indicator (which is just the probability it's 1).
3. Sum them up. No need to worry about correlations.

Why It Works

From the definition of expectation:

$$\begin{aligned} E[X + Y] &= \sum_{x,y} (x + y)P(X = x, Y = y) = \sum_{x,y} xP(X = x, Y = y) + \sum_{x,y} yP(X = x, Y = y) \\ &= \sum_x x \sum_y P(X = x, Y = y) + \sum_y y \sum_x P(X = x, Y = y) = \sum_x xP(X = x) + \sum_y yP(Y = y) = E[X] + E[Y] \end{aligned}$$

The double sum separates because of the distributive law, not independence.

3. When to Use It

- Problems asking "expected number of something" where the something is a count
- Problems where tracking dependencies between events is hard
- Analyzing randomized algorithms (expected swaps, comparisons)

- Computing expected number of fixed points in a permutation
- Expected number of distinct elements in a random selection
- Any problem where you can decompose a random variable into a sum of indicators

4. When Not to Use It

- When you need the distribution (not just the expectation)
- When you need expected value of a product or ratio ($E[XY] \neq E[X]E[Y]$ generally)
- When the decomposition into indicators is unnatural
- When the problem asks for variance or higher moments

5. Core Concepts

Indicator Random Variable

A variable that is 1 if an event occurs, 0 otherwise.

$$I_A = \begin{cases} 1 & \text{if event A occurs} \\ 0 & \text{otherwise} \end{cases}$$

$$E[I_A] = P(A)$$

Key Formula

If $X = X_1 + X_2 + \dots + X_n$, then:

$$E[X] = E[X_1] + E[X_2] + \dots + E[X_n]$$

No Independence Needed

This is the crucial point. You can sum expectations even when variables are correlated.

6. Step-by-Step Algorithm

To use linearity of expectation:

1. Identify the random variable X you need $E[X]$ for.
2. Decompose X into a sum of simpler random variables (often indicators).
3. Compute E [each simpler variable].
4. Sum them up.

7. Dry Run

Problem: A random permutation of $\{1, 2, 3, \dots, n\}$ is chosen uniformly. What is the expected number of fixed points (elements that stay in their original position)?

Step 1: Let X = number of fixed points.

Step 2: Decompose: $X = I_1 + I_2 + \dots + I_n$, where $I_i = 1$ if element i is at position i .

Step 3: For any i , $P(I_i = 1) = 1/n$ (element i is equally likely to be in any position).

Step 4: $E[I_i] = 1/n$.

Step 5: $E[X] = E[I_1] + \dots + E[I_n] = n \times (1/n) = 1$.

Answer: Expected number of fixed points = 1 (for any $n \geq 1$).

i	$P(I_i = 1)$	$E[I_i]$
1	$1/n$	$1/n$
2	$1/n$	$1/n$
...	...	...
n	$1/n$	$1/n$
Total		1

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Expected number of fixed points in a random permutation of size n
double expectedFixedPoints(int n) {
    // Each of n elements has 1/n chance of being fixed
    // E[X] = n * (1/n) = 1
    return 1.0; // Always 1 for any n >= 1
}

// Expected number of distinct coupons after k draws (with replacement, n coupons)
double expectedDistinct(int n, int k) {
    // Indicator for each coupon: I_i = 1 if coupon i is drawn at least once
    // P(I_i = 1) = 1 - ((n-1)/n)^k
    // E[distinct] = n * (1 - ((n-1)/n)^k)
    return n * (1.0 - pow((double)(n-1)/n, k));
}

// Expected number of trials to get 6 on a die (using linearity isn't ideal here, but for demo)
double expectedRollsForSix() {
    // Geometric distribution with p = 1/6
    return 6.0;
}

int main() {
    cout << "Expected fixed points (n=10): " << expectedFixedPoints(10) << "\n";
    cout << "Expected distinct coupons (n=10, k=5): " << expectedDistinct(10, 5) << "\n";
    cout << "Expected rolls for a six: " << expectedRollsForSix() << "\n";
    return 0;
}
```

9. Python Implementation

```
def expected_fixed_points(n: int) -> float:
    return 1.0 # Always 1

def expected_distinct(n: int, k: int) -> float:
    return n * (1.0 - ((n - 1) / n) ** k)

def expected_rolls_for_six() -> float:
    return 6.0
```

```
print(f"Expected fixed points (n=10): {expected_fixed_points(10)}")
print(f"Expected distinct coupons (n=10, k=5): {expected_distinct(10, 5)}")
print(f"Expected rolls for a six: {expected_rolls_for_six()}")
```

10. Code Explanation

- `expectedFixedPoints`: Demonstrates that the answer is always 1 regardless of n . This is a classic linearity result.
- `expectedDistinct`: For each coupon, probability it's never drawn = $((n-1)/n)^k$. Complement gives probability it's drawn at least once. Sum over n coupons.
- Both use indicator variables + linearity. No need to track correlations.

11. Complexity Analysis

Operation	Time	Space
Fixed points (any n)	$O(1)$	$O(1)$
Distinct coupons	$O(1)$	$O(1)$
General linearity problem (n indicators)	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identification	Decomposition	Example
Fixed points	"expected number of elements that stay"	I_i = element i is in correct position	Permutation problems
Distinct items	"expected number of different types"	I_i = type i is present	Coupon collector, birthday problem
Matchings/assignments	"expected pairs that match"	$I_{\{i,j\}}$ = pair (i,j) matches	Matching problems
Random graph edges	"expected number of edges"	$I_{\{u,v\}}$ = edge exists	Random graph properties
Random swaps in sorting	"expected number of comparisons"	$I_{\{i,j\}}$ = elements i,j compared	Randomized quicksort analysis

13. Common Mistakes

- Forgetting that linearity works for ANY random variables, not just independent ones.
- Trying to compute $E[XY]$ using linearity (wrong — need independence or covariance).
- Not properly defining indicator variables.
- Computing $P(I_i = 1)$ incorrectly due to counting errors.
- Applying linearity when the problem asks for variance or probability of an event.

14. Edge Cases

- $n = 0$ or $n = 1$ for permutations.
- $k = 0$ for coupon collector (distinct = 0).
- When probabilities are very small, use `log1p` or `expm1` for numerical stability.
- When n is large and k is small, be careful with floating point.

15. Variations

Variation	What Changes	When Used	Importance
Non-uniform indicators	Different probabilities for each	Weighted random selection	Moderate
Repeated applications	Use multiple times in sequence	Complex problems	High
Combined with DP	State-based indicator expectation	Expected value DP	High for CP

16. Related Concepts

- **Expected Value:** The base concept.
- **Indicator Variables:** The building blocks for linearity.
- **Variance:** Linearity does NOT apply to variance (needs independence).
- **Coupon Collector:** Classic application of linearity.

17. Practice Problems

Easy

- **Expected Number of Fixed Points** (GFG) — Direct application.
- **Birthday Paradox Expectation** (LeetCode) — Expected distinct birthdays.

Medium

- **Random Flip Matrix** (LeetCode 519) — Expected operations.
- **Random Pick Index** (LeetCode 398) — Expected comparisons.
- **Sereja and Swaps** (Codeforces) — Expected value after swaps.

Hard

- **Randomly Generated Permutations** (Codeforces) — Combined linearity + DP.
- **Probability of a Random Walk** (CSES) — Expectation in random processes.

18. Interview Explanation

"Linearity of expectation is the principle that the expected value of a sum equals the sum of expected values, with no independence condition needed. This is incredibly powerful. I use it by decomposing a complex random variable into indicator variables — each indicating whether a particular event occurs. The expected value of each indicator is just the probability of that event. I sum these probabilities, and I'm done. For example, the expected number of fixed points in a random permutation is always 1, regardless of the size of the permutation."

19. Revision Notes

- $E[X + Y] = E[X] + E[Y]$ (always)
- Decompose into indicators: $I_A = 1$ if A occurs
- $E[I_A] = P(A)$
- No independence required
- Most common trick: "expected count" = sum of probabilities
- Does NOT work for variance, products, or ratios

20. Final Cheat Sheet

Principle	Formula	Use
Linearity	$E[X+Y] = E[X] + E[Y]$	Always true
Indicator	$E[I_A] = P(A)$	Building block
Expected count	$E[\text{count}] = \sum P(\text{event } i)$	Most common pattern
Fixed points	Always 1	Classic example
Distinct items	$n \times (1 - ((n-1)/n)^k)$	Coupon collector variant

RANDOM VARIABLES

1. Overview

A random variable is a variable whose value depends on the outcome of a random experiment. It assigns a numerical value to each outcome in the sample space. Random variables are the bridge between probability theory and real-world measurements.

2. Intuition

Simple Explanation

Before you roll a die, the result is unknown — it's a random variable X . After the roll, X takes a specific value (say, 4). A random variable is not a single number; it's a function from the sample space to real numbers.

Analogy: The Lottery Ticket

A lottery ticket has a random value. Before the drawing, its value is a random variable: \$1000 with probability 0.001, \$0 with probability 0.999. After the drawing, it becomes a fixed number. The random variable is the "before" state.

Step-by-Step Reasoning

1. Define the sample space of an experiment.
2. Define a function that maps each outcome to a number.
3. That function is a random variable.
4. The distribution of the random variable describes how likely each value is.

Why It Works

Random variables allow us to talk about numerical properties of random experiments — averages, variances, probabilities of ranges — without listing every outcome explicitly.

3. When to Use It

- Any problem involving random numerical outcomes
 - Defining distributions for expected value calculations
 - Analyzing randomized algorithms
 - Problems with "probability that $X > k$ " or "expected value of X "
 - Monte Carlo simulations
-

4. When Not to Use It

- When the outcome is deterministic (no randomness)
- When you only need categorical outcomes (use events instead)

- When the problem is purely combinatorial without randomness

5. Core Concepts

Discrete Random Variable

Takes countably many values. Example: number of heads in 3 coin flips.

Continuous Random Variable

Takes values from a continuous range. Example: exact time of the next bus arrival.

Probability Mass Function (PMF) — Discrete

$P(X = x)$ = probability that X takes value x . Sums to 1.

Probability Density Function (PDF) — Continuous

$f(x)$ such that $P(a \leq X \leq b) = \int_a^b f(x) dx$. Integrates to 1.

Cumulative Distribution Function (CDF)

$F(x) = P(X \leq x)$. Works for both discrete and continuous.

Support

The set of values where the PMF/PDF is non-zero.

6. Step-by-Step Algorithm

To work with a discrete random variable:

1. Define the sample space of the experiment.
2. Define the mapping from outcomes to numeric values.
3. Compute the probability of each possible value.
4. Verify probabilities sum to 1.
5. Use the PMF to compute expectations, variance, probabilities.

7. Dry Run

Problem: Let X = sum of two fair dice. Find $P(X = 7)$.

Step 1: Sample space = 36 ordered pairs.

Step 2: X = sum of the two numbers.

Step 3: Count outcomes for each sum.

$X = \text{sum}$	Outcomes	Count	$P(X)$
2	(1,1)	1	1/36
3	(1,2),(2,1)	2	2/36
4	(1,3),(2,2),(3,1)	3	3/36
5	(1,4),(2,3),(3,2),(4,1)	4	4/36
6	(1,5),(2,4),(3,3),(4,2),(5,1)	5	5/36

X = sum	Outcomes	Count	P(X)
7	(1,6),(2,5),(3,4),(4,3),(5,2),(6,1)	6	6/36
8	(2,6),(3,5),(4,4),(5,3),(6,2)	5	5/36
9	(3,6),(4,5),(5,4),(6,3)	4	4/36
10	(4,6),(5,5),(6,4)	3	3/36
11	(5,6),(6,5)	2	2/36
12	(6,6)	1	1/36

$P(X = 7) = 6/36 = 1/6$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Compute PMF of sum of two n-sided dice
vector<double> twoDicePMF(int n) {
    // Possible sums: 2 to 2n
    int maxSum = 2 * n;
    vector<double> pmf(maxSum + 1, 0.0);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            pmf[i + j] += 1.0;
        }
    }

    double total = n * n;
    for (int s = 2; s <= maxSum; s++) {
        pmf[s] /= total;
    }

    return pmf;
}

// Compute CDF from PMF
vector<double> computeCDF(const vector<double>& pmf) {
    vector<double> cdf(pmf.size(), 0.0);
    double running = 0.0;
    for (size_t i = 0; i < pmf.size(); i++) {
        running += pmf[i];
        cdf[i] = running;
    }
    return cdf;
}

// Verify PMF sums to 1
double checkPMF(const vector<double>& pmf) {
    return accumulate(pmf.begin(), pmf.end(), 0.0);
}

int main() {
    auto pmf = twoDicePMF(6);
    cout << "P(sum = 7): " << pmf[7] << "\n";
}
```

```

cout << "PMF sums to: " << checkPMF(pmf) << "\n";

auto cdf = computeCDF(pmf);
cout << "P(sum <= 7): " << cdf[7] << "\n";

return 0;
}

```

9. Python Implementation

```

def two_dice_pmf(n: int) -> list:
    max_sum = 2 * n
    pmf = [0.0] * (max_sum + 1)

    for i in range(1, n + 1):
        for j in range(1, n + 1):
            pmf[i + j] += 1.0

    total = n * n
    for s in range(2, max_sum + 1):
        pmf[s] /= total

    return pmf

def compute_cdf(pmf: list) -> list:
    cdf = [0.0] * len(pmf)
    running = 0.0
    for i, p in enumerate(pmf):
        running += p
        cdf[i] = running
    return cdf

pmf = two_dice_pmf(6)
print(f"P(sum = 7): {pmf[7]}")
print(f"PMF sums to: {sum(pmf)}")

cdf = compute_cdf(pmf)
print(f"P(sum <= 7): {cdf[7]}")

```

10. Code Explanation

- `twoDicePMF`: Counts all 36 outcomes, normalizes by total. Returns array where index = sum value.
- `computeCDF`: Cumulative sum of PMF. $CDF[x] = P(X \leq x)$.
- `checkPMF`: Verifies normalization. Should be 1.0 (within floating error).
- The PMF array is indexed by the value. For values that never occur (like 0, 1), the entry stays 0.

11. Complexity Analysis

Operation	Time	Space
PMF computation (two dice)	$O(n^2)$	$O(n)$
CDF from PMF	$O(n)$	$O(n)$

Operation	Time	Space
PMF verification	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identification	Approach	Example
Sum of random variables	"sum of two dice"	Convolution of PMFs	Dice problems
Maximum/minimum	"max of random variables"	Use CDF: $P(\max \leq k) = \prod P(X_i \leq k)$	Lifetime of parallel systems
Transformations	" $Y = g(X)$ "	Express $P(Y = y)$ in terms of $P(X = g^{-1}(y))$	Squaring a random variable
Mixture distributions	"choose from distribution A with probability p"	Weighted average of PMFs	Two-stage experiments

13. Common Mistakes

- Confusing PMF and CDF.
- Forgetting to normalize probabilities.
- Mixing up discrete and continuous random variables.
- Using PDF values as probabilities (PDF can be > 1).
- Assuming $P(X = x) = 0$ for continuous variables (it is, but PDF is not).

14. Edge Cases

- Zero probability values (include in support or not).
- Infinite support (geometric, Poisson).
- Degenerate random variable (constant with probability 1).
- PMF entries that don't sum to 1 due to floating point.

15. Variations

Variation	What Changes	When Used	Importance
Continuous random variable	PDF instead of PMF	Physical measurements, time	Moderate
Multivariate random variable	Joint distribution	Multiple correlated outcomes	Moderate
Mixed random variable	Part discrete, part continuous	Real-world phenomena	Low for CP

16. Related Concepts

- **Probability Distribution:** The full description of a random variable's behavior.
- **Expected Value:** The mean of the distribution.
- **Variance and Standard Deviation:** Measures of spread.
- **Moment Generating Functions:** Higher-level analysis tool.

17. Practice Problems

Easy

- **Binomial Distribution** (GFG) — Basic PMF computation.
- **Dice Roll Simulation** (LeetCode 1223) — PMF of dice sums.

Medium

- **Random Pick with Weight** (LeetCode 528) — Working with non-uniform discrete distributions.
- **Probability of a Random Variable** (Codeforces) — Distribution analysis.
- **Generating Random Numbers** (LeetCode) — Inverse transform sampling.

Hard

- **Random Point in Non-overlapping Rectangles** (LeetCode 497) — Complex distribution.
- **Probability of a Knight's Tour** — Advanced distribution analysis.

18. Interview Explanation

"A random variable is a function that assigns a numerical value to each outcome of a random experiment. For discrete random variables, we describe them using a probability mass function, which gives the probability of each possible value. For continuous variables, we use a probability density function. The key operations are computing expectations (weighted averages), probabilities of ranges (using the CDF), and transformations (functions of random variables). In coding, I often work with empirical distributions or compute probabilities by iterating over the sample space."

19. Revision Notes

- Random variable: function from sample space to $\mathbb{R}$
- PMF: $P(X = x)$ for discrete
- PDF: $f(x)$ for continuous ($P(a \leq X \leq b) = \int f(x) \, dx$)
- CDF: $F(x) = P(X \leq x)$
- $E[X] = \sum x \cdot P(X = x)$ for discrete
- $\text{Var}(X) = E[X^2] - (E[X])^2$

20. Final Cheat Sheet

Concept	Definition	Formula
Random Variable	Function: $\Omega \rightarrow \mathbb{R}$	$X(\omega)$
PMF (discrete)	$P(X = x)$	Sums to 1
PDF (continuous)	$f(x)$	Integrates to 1
CDF	$P(X \leq x)$	Monotonic, $[0,1]$
Expected value	Weighted average	$\sum x \cdot P(x)$ or $\int x \cdot f(x) \, dx$
Variance	Spread	$E[X^2] - (E[X])^2$

CONDITIONAL PROBABILITY

1. Overview

Conditional probability measures the probability of an event occurring given that another event has already occurred. It's the foundation of Bayesian thinking and is essential for problems involving sequential or dependent events.

2. Intuition

Simple Explanation

What's the probability it's raining given that you see wet umbrellas? That's a conditional probability: $P(\text{rain} \mid \text{wet umbrellas})$. The "given" part narrows down the sample space to only outcomes where the condition is true.

Analogy: The Filter

Imagine you have a bag of 10 red and 10 blue marbles. If you know that a randomly chosen marble is not blue, then you're only considering the red marbles. The probability it's red becomes 1 (certainty). Conditional probability is like applying a filter to the sample space.

Step-by-Step Reasoning

1. Start with the full sample space.
2. Apply the condition — only consider outcomes where the condition is true.
3. Renormalize: divide by the probability of the condition.
4. The resulting probability is the conditional probability.

Why It Works

The formula:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

This renormalizes the probability of $A \cap B$ by the probability of B , effectively restricting the sample space to B .

3. When to Use It

- Problems with sequential events (draw without replacement, multi-stage games)
 - Problems where partial information is known
 - "Probability of A given B" — explicitly stated
 - Medical testing, spam filtering, recommendation systems
 - Chain rule: $P(A \cap B) = P(A) \times P(B|A)$
-

4. When Not to Use It

- When events are independent ($P(A|B) = P(A)$, so conditional is unnecessary)
 - When you need the joint probability directly (use chain rule)
 - When the condition is not well-defined ($P(B) = 0$)
 - When the problem is simpler without conditioning
-

5. Core Concepts

Definition

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{provided } P(B) > 0$$

Chain Rule (Multiplication Rule)

$$P(A \cap B) = P(A) \times P(B|A) = P(B) \times P(A|B)$$

Extended:

$$P(A_1 \cap A_2 \cap \dots \cap A_n) = P(A_1) \times P(A_2|A_1) \times P(A_3|A_1 \cap A_2) \times \dots$$

Law of Total Probability

If $B_1, B_2, \dots, B_n$ partition the sample space:

$$P(A) = \sum_{i=1}^n P(A|B_i) \times P(B_i)$$

Independence

A and B are independent iff:

$$P(A|B) = P(A) \quad \text{or} \quad P(A \cap B) = P(A) \times P(B)$$

6. Step-by-Step Algorithm

To compute $P(A | B)$:

1. Verify $P(B) > 0$.
2. Compute $P(A \cap B)$ — the probability both A and B occur.
3. Compute $P(B)$ — the probability of the condition.
4. Divide: $P(A | B) = P(A \cap B) / P(B)$.

7. Dry Run

Problem: A bag has 3 red and 2 blue marbles. You draw two marbles without replacement. What is the probability the second marble is red given the first is red?

Step 1: Define events.

- A = second marble is red
- B = first marble is red

Step 2: Compute $P(B)$. $P(B) = 3/5$ (3 red out of 5 total).

Step 3: Compute $P(A \cap B)$. $P(A \cap B) = P(\text{first red}) \times P(\text{second red} | \text{first red}) = (3/5) \times (2/4) = 6/20 = 3/10$.

Step 4: Compute $P(A | B)$. $P(A | B) = (3/10) / (3/5) = (3/10) \times (5/3) = 1/2$.

State	Marbles	P(first red)	P(second red first red)
Initial	3R, 2B	3/5	—
After first red	2R, 2B	—	2/4 = 1/2

Answer: $P(\text{second red} | \text{first red}) = 1/2$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Conditional probability P(A|B) = P(A∩B) / P(B)
double conditionalProb(double pAandB, double pB) {
```

```

    if (pB == 0.0) return 0.0; // Undefined, return 0 as fallback
    return pAandB / pB;
}

// Chain rule:  $P(A \cap B) = P(A) * P(B|A)$ 
double chainRule(double pA, double pBgivenA) {
    return pA * pBgivenA;
}

// Law of total probability
// Given partitions  $B_i$  with probabilities  $pB[i]$  and  $P(A|B_i) = pAgivenB[i]$ 
double totalProbability(const vector<double>& pB, const vector<double>& pAgivenB) {
    assert(pB.size() == pAgivenB.size());
    double total = 0.0;
    for (size_t i = 0; i < pB.size(); i++) {
        total += pAgivenB[i] * pB[i];
    }
    return total;
}

int main() {
    // Two red marbles without replacement
    double pB = 3.0/5.0; // P(first red)
    double pAandB = (3.0/5.0) * (2.0/4.0); // P(both red)
    cout << "P(second red | first red): " << conditionalProb(pAandB, pB) << "\n";

    // Chain rule example
    cout << "P(both red) = " << chainRule(3.0/5.0, 2.0/4.0) << "\n";

    // Law of total probability example
    //  $P(\text{rain}) = P(\text{rain}|\text{sunny})P(\text{sunny}) + P(\text{rain}|\text{cloudy})P(\text{cloudy})$ 
    vector<double> pB = {0.7, 0.3}; // P(sunny), P(cloudy)
    vector<double> pAgivenB = {0.1, 0.8}; // P(rain|sunny), P(rain|cloudy)
    cout << "P(rain): " << totalProbability(pB, pAgivenB) << "\n";

    return 0;
}

```

9. Python Implementation

```

def conditional_prob(p_a_and_b: float, p_b: float) -> float:
    if p_b == 0.0:
        return 0.0
    return p_a_and_b / p_b

def chain_rule(p_a: float, p_b_given_a: float) -> float:
    return p_a * p_b_given_a

def total_probability(p_b: list, p_a_given_b: list) -> float:
    assert len(p_b) == len(p_a_given_b)
    return sum(p * p_given for p, p_given in zip(p_b, p_a_given_b))

# Two red marbles without replacement
p_b = 3.0 / 5.0 # P(first red)
p_a_and_b = (3.0 / 5.0) * (2.0 / 4.0) # P(both red)
print(f"P(second red | first red): {conditional_prob(p_a_and_b, p_b)}")

# Chain rule

```

```
print(f"P(both red): {chain_rule(3.0/5.0, 2.0/4.0)}")

# Law of total probability
print(f"P(rain): {total_probability([0.7, 0.3], [0.1, 0.8])}")
```

10. Code Explanation

- **conditionalProb**: Simple division with zero check. In CP, $P(B) = 0$ is rare but guard against it.
- **chainRule**: Multiplies along the dependency chain. Useful for sequential draws.
- **totalProbability**: Weighted average of conditional probabilities. The weights are the partition probabilities.
- All functions return **double**. Use **long double** if precision is critical.

11. Complexity Analysis

Operation	Time	Space
Single conditional probability	$O(1)$	$O(1)$
Chain rule (n events)	$O(n)$	$O(1)$
Law of total probability (n partitions)	$O(n)$	$O(1)$

12. Common Patterns

Pattern	Identification	Approach	Example
Without replacement	"draw without replacement"	Chain rule	Cards, marbles
Sequential trials	"first A then B"	Multiply conditional probabilities	Multi-stage experiments
Medical testing	"test positive given disease"	Conditional + Bayes	Sensitivity/specificity
Two-stage processes	"choose a box, then draw from it"	Law of total probability	Monty Hall, box problems

13. Common Mistakes

- Computing $P(A|B)$ when events are actually independent.
- Forgetting to renormalize (just using $P(A \cap B)$ without dividing by $P(B)$).
- Using $P(A|B) = P(B|A)$ — they are generally different.
- Dividing by zero when $P(B) = 0$.
- Not using the chain rule for sequential draws.

14. Edge Cases

- $P(B) = 0$ (conditional probability is undefined).
- $P(A|B) = 1$ when A is certain given B.
- $P(A|B) = 0$ when A is impossible given B.
- Events are independent: $P(A|B) = P(A)$.

15. Variations

Variation	What Changes	When Used	Importance
Conditional expectation	$E[X \mid Y = y]$	Advanced expectation	High
Conditional variance	$\text{Var}(X \mid Y)$	Risk analysis	Moderate
Conditional independence	$A \perp B \mid C$	Graphical models, Naive Bayes	High for ML

16. Related Concepts

- **Bayes Theorem:** Directly uses conditional probability to reverse the conditioning.
- **Law of Total Probability:** Expresses $P(A)$ as weighted sum of conditional probabilities.
- **Independence:** Special case where conditioning doesn't change probability.
- **Chain Rule:** Generalizes to multiple events.

17. Practice Problems

Easy

- **Probability of Drawing Marbles** (GFG) — Basic conditional probability.
- **Cards Without Replacement** (LeetCode) — Sequential conditional probability.

Medium

- **Probability of a Two Boxes Having Same Number of Distinct Balls** (LeetCode) — Complex conditional probability.
- **Monty Hall Problem** (GFG) — Classic conditional probability puzzle.
- **Random Pick with Blacklist** (LeetCode 710) — Conditional probability in sampling.

Hard

- **Probability of a Knight's Tour** — Complex conditional analysis.
- **New 21 Game** (LeetCode 837) — DP with conditional probability.

18. Interview Explanation

"Conditional probability measures the probability of an event given that another event has occurred. The formula is $P(A|B) = P(A \cap B) / P(B)$. It essentially restricts the sample space to outcomes where B is true and renormalizes. The chain rule extends this to multiple events: $P(A \cap B \cap C) = P(A) \times P(B|A) \times P(C|A \cap B)$. The law of total probability lets me compute $P(A)$ by considering different scenarios. These are essential for problems with sequential or dependent events."

19. Revision Notes

- $P(A|B) = P(A \cap B) / P(B)$, $P(B) > 0$
- Chain rule: $P(A \cap B) = P(A) \times P(B|A)$
- Law of total probability: $P(A) = \sum P(A|B_i) \times P(B_i)$
- Independent: $P(A|B) = P(A)$
- $P(A|B) \neq P(B|A)$ generally
- Without replacement $\rightarrow$ chain rule is natural

20. Final Cheat Sheet

Formula	Use	Example
$P(A B) = P(A \cap B) / P(B)$	Conditional probability	Drawing cards

Formula	Use	Example
$P(A \cap B) = P(A) \times P(B A)$	Chain rule	Sequential draws
$P(A) = \sum P(A B_i)P(B_i)$	Total probability	Multi-stage problems
$P(A B) = P(A)$	Independence check	Coin flips

BAYES THEOREM

1. Overview

Bayes Theorem describes the probability of an event based on prior knowledge of conditions that might be related to the event. It allows us to "reverse" conditional probability: given $P(B|A)$, compute $P(A|B)$.

2. Intuition

Simple Explanation

If you know that 1% of people have a disease, and the test is 99% accurate, what's the probability you actually have the disease if you test positive? Most people guess 99%, but Bayes shows it's much lower (around 50% for realistic numbers). Bayes tells us to update our beliefs based on new evidence.

Analogy: The News Update

You think there's a 30% chance it will rain today (prior). Then you see dark clouds (evidence). Your updated belief (posterior) is higher. Bayes Theorem quantifies this update.

Step-by-Step Reasoning

1. Start with a prior belief $P(A)$.
2. Observe evidence B .
3. Know how likely the evidence is if A is true: $P(B|A)$.
4. Know how likely the evidence is in general: $P(B)$.
5. Update: $P(A|B) = P(B|A) \times P(A) / P(B)$.

Why It Works

Bayes Theorem follows directly from the definition of conditional probability:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B|A) \times P(A)}{P(B)}$$

It's a mathematical identity, not an assumption.

3. When to Use It

- Problems where you need to reverse conditional probability
- Medical testing, spam filtering, classification
- "Given that B happened, what's the probability A caused it?"
- Problems with false positives and false negatives
- Naive Bayes classifiers in ML
- Problems with prior and posterior probabilities

4. When Not to Use It

- When you directly have $P(A|B)$ and don't need to reverse
 - When the problem is simpler without the Bayesian framework
 - When $P(B)$ is hard to compute and alternative approaches exist
 - When the prior is completely unknown and can't be reasonably estimated
-

5. Core Concepts

Bayes Formula

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Denominator (Normalization)

$$P(B) = P(B|A) \times P(A) + P(B|\neg A) \times P(\neg A)$$

Prior

$P(A)$ — your belief before seeing evidence.

Posterior

$P(A|B)$ — your updated belief after seeing evidence.

Likelihood

$P(B|A)$ — how likely the evidence is if your hypothesis is true.

Evidence

$P(B)$ — the overall probability of the evidence.

6. Step-by-Step Algorithm

To compute $P(A|B)$ using Bayes Theorem:

1. Identify the hypothesis A and the evidence B .
 2. Determine $P(A)$ — the prior probability.
 3. Determine $P(B|A)$ — the likelihood (true positive rate).
 4. Determine $P(B|\neg A)$ — the false positive rate.
 5. Compute $P(B) = P(B|A) \times P(A) + P(B|\neg A) \times P(\neg A)$.
 6. Compute $P(A|B) = P(B|A) \times P(A) / P(B)$.
-

7. Dry Run

Problem: A disease affects 1% of the population. A test is 99% accurate (99% sensitivity, 99% specificity). If a person tests positive, what's the probability they actually have the disease?

Step 1: Let A = has disease, B = tests positive.

Step 2: $P(A) = 0.01$ (prior).

Step 3: $P(B|A) = 0.99$ (true positive rate).

Step 4: $P(B|\neg A) = 0.01$ (false positive rate = $1 - \text{specificity}$).

Step 5: $P(B) = 0.99 \times 0.01 + 0.01 \times 0.99 = 0.0099 + 0.0099 = 0.0198$.

Step 6: $P(A|B) = 0.99 \times 0.01 / 0.0198 = 0.0099 / 0.0198 = 0.5$.

Quantity	Symbol	Value
Prior	$P(A)$	0.01
True positive rate	$P(B A)$	0.99
False positive rate	$P(B \neg A)$	0.01
$P(\text{positive})$	$P(B)$	0.0198
Posterior	$P(A B)$	0.5

Answer: Only 50% chance of actually having the disease despite 99% accurate test.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Bayes Theorem:  $P(A|B) = P(B|A) * P(A) / P(B)$ 
// where  $P(B) = P(B|A)*P(A) + P(B|\text{not}A)*P(\text{not}A)$ 
double bayesTheorem(double prior, double truePositiveRate, double falsePositiveRate) {
    double pB = truePositiveRate * prior + falsePositiveRate * (1.0 - prior);
    if (pB == 0.0) return 0.0;
    return (truePositiveRate * prior) / pB;
}

// Extended Bayes for multiple hypotheses
// Given: prior[i], likelihood[i] =  $P(\text{evidence} | \text{hypothesis } i)$ 
// Returns: posterior[i] =  $P(\text{hypothesis } i | \text{evidence})$ 
vector<double> bayesMultiple(const vector<double>& prior, const vector<double>& likelihood) {
    int n = prior.size();
    vector<double> posterior(n);
    double evidence = 0.0;

    for (int i = 0; i < n; i++) {
        evidence += likelihood[i] * prior[i];
    }

    if (evidence == 0.0) return posterior; // All zeros

    for (int i = 0; i < n; i++) {
        posterior[i] = (likelihood[i] * prior[i]) / evidence;
    }

    return posterior;
}

int main() {
    // Disease testing example
    double prior = 0.01;
    double tpr = 0.99; // Sensitivity
    double fpr = 0.01; // 1 - Specificity

    cout << "P(disease | positive): " << bayesTheorem(prior, tpr, fpr) << "\n";
}
```

```
// Multiple hypotheses: three boxes with different proportions of red/blue
// Box 1: 90% red, Box 2: 50% red, Box 3: 10% red
// Choose a box uniformly, draw a red ball. Which box is most likely?
vector<double> priorBox = {1.0/3, 1.0/3, 1.0/3};
vector<double> likelihood = {0.9, 0.5, 0.1}; // P(red | box i)

auto post = bayesMultiple(priorBox, likelihood);
cout << "Posterior probabilities: ";
for (double p : post) cout << p << " ";
cout << "\n";

return 0;
}
```

9. Python Implementation

```
def bayes_theorem(prior: float, true_positive_rate: float, false_positive_rate: float) -> float:
    p_evidence = true_positive_rate * prior + false_positive_rate * (1.0 - prior)
    if p_evidence == 0.0:
        return 0.0
    return (true_positive_rate * prior) / p_evidence

def bayes_multiple(prior: list, likelihood: list) -> list:
    n = len(prior)
    evidence = sum(likelihood[i] * prior[i] for i in range(n))
    if evidence == 0.0:
        return [0.0] * n
    return [(likelihood[i] * prior[i]) / evidence for i in range(n)]

# Disease testing
print(f"P(disease | positive): {bayes_theorem(0.01, 0.99, 0.01)}")

# Multiple hypotheses
prior_box = [1.0/3, 1.0/3, 1.0/3]
likelihood = [0.9, 0.5, 0.1]
post = bayes_multiple(prior_box, likelihood)
print(f"Posterior probabilities: {post}")
```

10. Code Explanation

- `bayesTheorem`: Direct implementation of Bayes formula. Computes $P(B)$ using law of total probability.
- `bayesMultiple`: Generalizes to n mutually exclusive hypotheses. Prior array must sum to 1.
- Both functions guard against zero evidence.
- The multiple hypothesis version is essentially the structure of Naive Bayes classifiers.

11. Complexity Analysis

Operation	Time	Space
Simple Bayes (2 hypotheses)	$O(1)$	$O(1)$
Multiple hypotheses (n)	$O(n)$	$O(n)$

Operation	Time	Space
Naive Bayes (n features, m classes)	$O(nm)$	$O(nm)$

12. Common Patterns

Pattern	Identification	Approach	Example
Medical testing	"test positive/negative", "disease"	Bayes with false positive/negative	Disease screening
Spam filtering	"given these words, is it spam?"	Naive Bayes	Email classification
Monty Hall	"given the host opened door X"	Update prior with evidence	Monty Hall problem
Source identification	"which box/bag/group did this come from?"	Multiple hypothesis Bayes	Box selection problems

13. Common Mistakes

- Forgetting the denominator (only computing numerator).
- Using $P(B|A)$ as the answer (confusing with $P(A|B)$).
- Ignoring the base rate (prior probability).
- Double counting evidence.
- Using Bayes when events are independent (unnecessary).

14. Edge Cases

- Prior = 0 or 1 (certainty, evidence doesn't change it).
- $P(B|A) = 0$ (evidence impossible under hypothesis).
- False positive rate = 0 (perfect test, posterior = 1).
- Very small probabilities causing floating point underflow.

15. Variations

Variation	What Changes	When Used	Importance
Sequential Bayes	Update repeatedly with new evidence	Real-time filtering	High
Naive Bayes classifier	Assumes feature independence	ML classification	High for ML
Bayesian inference	Continuous priors	Parameter estimation	Moderate

16. Related Concepts

- **Conditional Probability:** The foundation of Bayes.
- **Law of Total Probability:** Used to compute the denominator.
- **Prior vs Posterior:** Core Bayesian terminology.
- **Naive Bayes:** Practical ML application.

17. Practice Problems

Easy

- **Bayes Theorem** (GFG) — Basic application.

- **Probability of Disease** (LeetCode) — Medical testing problem.

Medium

- **Monty Hall Problem** (GFG) — Classic Bayes puzzle.
- **Random Pick with Blacklist** (LeetCode 710) — Conditional probability.
- **Spam Filter** (Codeforces) — Naive Bayes application.

Hard

- **Probability of a Random Walk** (CSES) — Complex Bayesian inference.
- **Bayesian Inference on Graphs** — Advanced application.

18. Interview Explanation

"Bayes Theorem is a formula for reversing conditional probability: $P(A|B) = P(B|A) \times P(A) / P(B)$. It's crucial in situations where we have a prior belief and observe evidence. The classic example is medical testing: even with a 99% accurate test, if a disease is rare (1% prevalence), a positive result only gives about 50% probability of actually having the disease. The key insight is to always account for the base rate through the denominator, which normalizes the probability."

19. Revision Notes

- $P(A|B) = P(B|A) \times P(A) / P(B)$
 - $P(B) = P(B|A)P(A) + P(B|\neg A)P(\neg A)$
 - Prior $\rightarrow$ Evidence $\rightarrow$ Posterior
 - Base rate fallacy: ignoring the prior
 - $P(A|B) \neq P(B|A)$ — don't confuse them
 - Sequential: posterior becomes new prior
-

20. Final Cheat Sheet

Component	Formula	Meaning
Posterior	$P(A B)$	Updated belief after evidence
Prior	$P(A)$	Initial belief
Likelihood	$P(B A)$	Evidence under hypothesis
Evidence	$P(B)$	Total probability of evidence
Bayes	$P(A B) = L \times P(A) / P(B)$	The full formula

MARKOV CHAINS

1. Overview

A Markov chain is a stochastic process where the future state depends only on the current state, not on the sequence of events that preceded it (the Markov property). It's a powerful model for random processes with limited memory.

2. Intuition

Simple Explanation

Think of a frog jumping between lily pads. Where the frog jumps next depends only on where it is now, not on how it got there. The system has no memory beyond the current state.

Analogy: The Weather

If today is sunny, tomorrow might be sunny with probability 0.8 or rainy with probability 0.2. The weather tomorrow depends only on today's weather, not on yesterday's. This is a Markov chain with states {Sunny, Rainy}.

Step-by-Step Reasoning

1. Define a set of states.
2. Define transition probabilities: given current state i , probability of moving to state j .
3. The process moves from state to state according to these probabilities.
4. The Markov property says: the next state depends only on the current state.

Why It Works

Markov chains are memoryless by design. This makes them computationally tractable while still modeling many real-world processes. The transition matrix captures all the dynamics.

3. When to Use It

- Modeling random processes with sequential states
- Analyzing random walks on graphs
- PageRank algorithm (Google's original ranking)
- Predicting weather, stock prices, queue lengths
- Problems with "states" and "transitions" and "probability of reaching state X "
- Expected hitting time problems
- Any problem where the future depends only on the present

4. When Not to Use It

- When the process has long-term memory (non-Markovian)
- When the state space is too large to enumerate
- When transitions are not time-homogeneous (probabilities change over time)
- When the process is deterministic (use a regular graph/DP instead)
- When you need exact path probabilities (use tree-based models)

5. Core Concepts

States

The possible configurations of the system. Usually numbered $0, 1, \dots, n-1$.

Transition Matrix (P)

An $n \times n$ matrix where $P[i][j]$ = probability of moving from state i to state j in one step. Each row sums to 1.

Markov Property

$$P(X_{t+1} = j \mid X_t = i, X_{t-1}, \dots, X_0) = P(X_{t+1} = j \mid X_t = i)$$

Initial Distribution (π_0)

The probability of starting in each state.

Stationary Distribution (π)

A distribution such that $\pi = \pi P$. After many steps, the chain converges to this distribution.

Absorbing State

A state that, once entered, cannot be left. $P(\text{absorbing} \rightarrow \text{absorbing}) = 1$.

Transient State

A state that is not absorbing. The chain may leave it.

6. Step-by-Step Algorithm

To simulate a Markov chain for T steps:

1. Define the transition matrix P ($n \times n$).
2. Start at an initial state (or sample from initial distribution).
3. For each step $t = 1$ to T : a. Generate a random number r in $[0, 1)$. b. Use the current state's row in P to determine the next state. c. Move to the next state.
4. Record the sequence of states.

To compute the distribution after k steps:

1. Let π_0 be the initial distribution (row vector).
2. Compute $\pi_k = \pi_0 \times P^k$.

7. Dry Run

Problem: A simple 2-state Markov chain for weather.

- State 0: Sunny, State 1: Rainy
- Transition matrix:
 - From Sunny: $P(\text{Sunny}|\text{Sunny}) = 0.8$, $P(\text{Rainy}|\text{Sunny}) = 0.2$
 - From Rainy: $P(\text{Sunny}|\text{Rainy}) = 0.4$, $P(\text{Rainy}|\text{Rainy}) = 0.6$

Initial state: Sunny (Day 0).

Step 1: Day 1:

- From Sunny: $P(\text{Sunny}) = 0.8$, $P(\text{Rainy}) = 0.2$
- Random number 0.35 $\rightarrow$ Sunny (since $0.35 < 0.8$)

Step 2: Day 2:

- From Sunny: $P(\text{Sunny}) = 0.8$, $P(\text{Rainy}) = 0.2$
- Random number 0.91 $\rightarrow$ Rainy (since $0.91 \geq 0.8$)

Step 3: Day 3:

- From Rainy: $P(\text{Sunny}) = 0.4$, $P(\text{Rainy}) = 0.6$
- Random number 0.50 $\rightarrow$ Rainy (since $0.50 \geq 0.4$)

Day	State	Random #	Next State
0	Sunny	—	—
1	Sunny	0.35	Sunny

Day	State	Random #	Next State
2	Sunny	0.91	Rainy
3	Rainy	0.50	Rainy

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class MarkovChain {
private:
    vector<vector<double>> trans; // transition matrix
    int n; // number of states
    mt19937 rng;

public:
    MarkovChain(const vector<vector<double>>& transitionMatrix)
        : trans(transitionMatrix), n(transitionMatrix.size()),
        rng(chrono::steady_clock::now().time_since_epoch().count()) {
        // Validate: each row should sum to ~1
        for (int i = 0; i < n; i++) {
            double sum = accumulate(trans[i].begin(), trans[i].end(), 0.0);
            if (abs(sum - 1.0) > 1e-9) {
                cerr << "Warning: Row " << i << " sums to " << sum << "\n";
            }
        }
    }

    // Simulate one step from current state
    int nextState(int current) {
        double r = uniform_real_distribution<double>(0.0, 1.0)(rng);
        double cum = 0.0;
        for (int j = 0; j < n; j++) {
            cum += trans[current][j];
            if (r < cum) return j;
        }
        return n - 1; // Fallback due to floating point
    }

    // Simulate for T steps starting from startState
    vector<int> simulate(int startState, int steps) {
        vector<int> path;
        int state = startState;
        path.push_back(state);
        for (int t = 0; t < steps; t++) {
            state = nextState(state);
            path.push_back(state);
        }
        return path;
    }

    // Compute distribution after k steps from initial distribution
    vector<double> distributionAfter(const vector<double>& initDist, int k) {
        vector<double> dist = initDist;
        for (int step = 0; step < k; step++) {
            vector<double> next(n, 0.0);
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {

```

```

        next[j] += dist[i] * trans[i][j];
    }
}
dist = next;
}
return dist;
}

// Compute stationary distribution by power iteration
vector<double> stationaryDistribution(int maxIter = 10000, double tol = 1e-12) {
    vector<double> dist(n, 1.0 / n);
    for (int iter = 0; iter < maxIter; iter++) {
        vector<double> next(n, 0.0);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                next[j] += dist[i] * trans[i][j];
            }
        }
        double diff = 0.0;
        for (int i = 0; i < n; i++) diff += abs(next[i] - dist[i]);
        dist = next;
        if (diff < tol) break;
    }
    return dist;
}

};

int main() {
    // Weather: 0=Sunny, 1=Rainy
    vector<vector<double>> weather = {
        {0.8, 0.2},
        {0.4, 0.6}
    };

    MarkovChain mc(weather);

    // Simulate 10 days starting from Sunny
    auto path = mc.simulate(0, 10);
    cout << "Weather path (0=Sunny, 1=Rainy): ";
    for (int s : path) cout << s << " ";
    cout << "\n";

    // Distribution after 5 days starting from Sunny
    auto dist5 = mc.distributionAfter({1.0, 0.0}, 5);
    cout << "After 5 days: P(Sunny)=" << dist5[0] << " P(Rainy)=" << dist5[1] << "\n";

    // Stationary distribution
    auto stat = mc.stationaryDistribution();
    cout << "Stationary: P(Sunny)=" << stat[0] << " P(Rainy)=" << stat[1] << "\n";

    return 0;
}

```

9. Python Implementation

```

import random
from typing import List

```

```

class MarkovChain:
    def __init__(self, transition_matrix: List[List[float]]):
        self.trans = transition_matrix
        self.n = len(transition_matrix)
        # Validate rows sum to ~1
        for i, row in enumerate(self.trans):
            if abs(sum(row) - 1.0) > 1e-9:
                print(f"Warning: Row {i} sums to {sum(row)}")

    def next_state(self, current: int) -> int:
        r = random.random()
        cum = 0.0
        for j in range(self.n):
            cum += self.trans[current][j]
            if r < cum:
                return j
        return self.n - 1

    def simulate(self, start_state: int, steps: int) -> List[int]:
        path = [start_state]
        state = start_state
        for _ in range(steps):
            state = self.next_state(state)
            path.append(state)
        return path

    def distribution_after(self, init_dist: List[float], k: int) -> List[float]:
        dist = init_dist[:]
        for _ in range(k):
            nxt = [0.0] * self.n
            for i in range(self.n):
                for j in range(self.n):
                    nxt[j] += dist[i] * self.trans[i][j]
            dist = nxt
        return dist

    def stationary_distribution(self, max_iter: int = 10000, tol: float = 1e-12) ->
List[float]:
        dist = [1.0 / self.n] * self.n
        for _ in range(max_iter):
            nxt = [0.0] * self.n
            for i in range(self.n):
                for j in range(self.n):
                    nxt[j] += dist[i] * self.trans[i][j]
            diff = sum(abs(nxt[i] - dist[i]) for i in range(self.n))
            dist = nxt
            if diff < tol:
                break
        return dist

# Weather example
weather = [
    [0.8, 0.2],
    [0.4, 0.6]
]

mc = MarkovChain(weather)

path = mc.simulate(0, 10)
print(f"Weather path: {path}")

dist5 = mc.distribution_after([1.0, 0.0], 5)

```

```
print(f"After 5 days: P(Sunny)={dist5[0]:.4f} P(Rainy)={dist5[1]:.4f}")

stat = mc.stationary_distribution()
print(f"Stationary: P(Sunny)={stat[0]:.4f} P(Rainy)={stat[1]:.4f}")
```

10. Code Explanation

- **MarkovChain** class: Encapsulates the transition matrix and state count.
- **nextState**: Uses inverse transform sampling on the current state's row.
- **simulate**: Generates a path by repeatedly calling **nextState**.
- **distributionAfter**: Computes $\pi_k = \pi_0 \times P^k$ via matrix-vector multiplication.
- **stationaryDistribution**: Iterates $\pi \leftarrow \pi P$ until convergence (power iteration).
- Constructor validates that rows sum to 1 (with tolerance).

11. Complexity Analysis

Operation	Time	Space
One step simulation	$O(n)$	$O(1)$
T-step simulation	$O(T \cdot n)$	$O(T)$
Distribution after k steps	$O(k \cdot n^2)$	$O(n)$
Stationary distribution (power iteration)	$O(\text{iter} \cdot n^2)$	$O(n)$
Transition matrix storage	—	$O(n^2)$

12. Common Patterns

Pattern	Identification	Approach	Example
Random walk on graph	"particle moving on graph"	States = nodes, transitions = edges	Random walk on a grid
Absorbing Markov chain	"reach a goal state and stop"	Gambler's ruin, PageRank	Hitting time problems
Hidden Markov Model	"observations generated by hidden states"	Viterbi algorithm, Forward-backward	Speech recognition
Expected hitting time	"expected steps to reach state X"	Solve linear equations: $E[i] = 1 + \sum P[i \rightarrow j] \cdot E[j]$	Snakes and Ladders
Periodic behavior	"cycles in the chain"	Period analysis	Card shuffling

13. Common Mistakes

- Forgetting that rows (not columns) of the transition matrix must sum to 1.
- Assuming all Markov chains have a unique stationary distribution (needs irreducibility + aperiodicity).
- Not checking for absorbing states.
- Using Markov chain when the process has memory beyond the current state.
- Confusing the transition matrix with its transpose.
- Incorrectly computing distribution after k steps (left multiplication vs right).

14. Edge Cases

- Single state chain (trivial, always stays in that state).
- Deterministic transitions (all probabilities are 0 or 1).
- Periodic chain (e.g., alternating between two states).
- Reducible chain (multiple disconnected components).
- Chain with absorbing states.

15. Variations

Variation	What Changes	When Used	Importance
Absorbing Markov chain	Some states are absorbing	Hitting time, gambler's ruin	High
Continuous-time Markov chain	Continuous time instead of discrete steps	Queueing theory	Moderate
Hidden Markov Model	States are hidden, observations are visible	Speech recognition, bioinformatics	High for ML
Markov Decision Process	Actions affect transitions	Reinforcement learning	High for ML

16. Related Concepts

- **Random Walk:** A special case of Markov chain on a graph.
- **Stationary Distribution:** The long-run behavior of the chain.
- **PageRank:** Uses Markov chain theory for web ranking.
- **Monte Carlo Methods:** Use Markov chains for sampling (MCMC).

17. Practice Problems

Easy

- **Simulate a Markov Chain** (GFG) — Basic simulation.
- **Weather Prediction** (Codeforces) — Simple 2-state chain.

Medium

- **Snakes and Ladders** (LeetCode 909) — Markov chain analysis for expected moves.
- **Random Walk on a Graph** (CSES) — Expected hitting time.
- **PageRank Simulation** (Codeforces) — Basic PageRank.

Hard

- **Expected Number of Steps to Reach State** (Codeforces) — System of linear equations for absorbing chain.
- **Hidden Markov Model** (LeetCode) — Viterbi algorithm.
- **MCMC Sampling** (Advanced) — Metropolis-Hastings algorithm.

18. Interview Explanation

"A Markov chain is a stochastic process where the next state depends only on the current state. It's defined by a transition matrix where each row sums to 1. The key result is the stationary distribution — the long-run proportion of time spent in each state. For absorbing chains, we can compute expected hitting times by solving a system of linear equations. In coding problems, Markov chains appear in random walks on graphs, expected time to reach a target state, and PageRank-style algorithms."

19. Revision Notes

- Markov property: future depends only on present
- Transition matrix P : $P[i][j] = P(\text{next} = j \mid \text{current} = i)$
- Row sums = 1
- Distribution after k steps: $\pi_k = \pi_0 \times P^k$
- Stationary distribution: $\pi = \pi P$
- Absorbing state: $P[i][i] = 1$
- Hitting time: expected steps to reach a target state
- Needs irreducibility + aperiodicity for unique stationary distribution

20. Final Cheat Sheet

Concept	Formula/Definition	Use
Markov property	$P(X_{t+1} \mid X_t, \dots, X_0) = P(X_{t+1} \mid X_t)$	Memoryless
Transition matrix	$P[i][j] = P(\text{next}=j \mid \text{current}=i)$	Row sums to 1
k-step distribution	$\pi_k = \pi_0 \times P^k$	Future prediction
Stationary distribution	$\pi = \pi P$	Long-run behavior
Hitting time	$E[H] = (I - Q)^{-1} \cdot 1$	Absorbing chains
Power iteration	$\pi \leftarrow \pi P$	Computing stationary dist

COUPON COLLECTOR

1. Overview

The Coupon Collector problem asks: if there are n distinct coupons (types) and you get one random coupon each time, how many draws do you expect to collect all n types? It's a classic application of expectation and a fundamental result in probability.

2. Intuition

Simple Explanation

A cereal brand releases 5 different toy types. Each box contains one random toy. How many boxes do you need to buy to collect all 5? The answer isn't 5 — it's about 11.4 on average. The last few toys are the hardest to find.

Analogy: The Sticker Album

Collecting World Cup stickers: the first few are easy (you almost always get new ones), but the last few stickers are incredibly rare (you have a $1/n$ chance of getting the one you're missing). The expected number of packs grows as $n \log(n)$.

Step-by-Step Reasoning

1. You start with 0 coupons.
2. To get the first coupon: you always get a new one. Expected draws = 1.
3. To get the second (different from the first): probability of getting a new one = $(n-1)/n$. Expected draws = $n/(n-1)$.
4. To get the third: probability = $(n-2)/n$. Expected draws = $n/(n-2)$.
5. Continue until the last coupon: probability = $1/n$. Expected draws = n .
6. Sum all expected draws: $n \cdot H_n$ (where H_n is the n th harmonic number).

Why It Works

The process is a sequence of geometric random variables. When you have k distinct coupons, the probability of getting a new one is $(n-k)/n$. The expected number of draws to get the next new coupon is $n/(n-k)$. By linearity of expectation, the total expected draws is the sum of these.

3. When to Use It

- Problems asking "expected number of trials to collect all items"
 - Problems involving "collecting sets", "complete the collection", "all types"
 - Probability problems with "draw with replacement until all seen"
 - Analysis of randomized algorithms (expected iterations to cover all cases)
 - Birthday paradox type problems
 - Problems about "how many random samples to see every element"
-

4. When Not to Use It

- When draws are without replacement (use hypergeometric)
 - When coupons have different probabilities (use weighted coupon collector)
 - When you need the distribution, not just the expectation
 - When the problem asks for the probability of completing in exactly k draws
 - When the number of coupons is very large ($n > 10^6$) and you need exact answer
-

5. Core Concepts

Expected Number of Draws

$$E[T] = n \times H_n = n \times \left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right)$$

Harmonic Number

$$H_n = \sum_{k=1}^n \frac{1}{k} \approx \ln n + \gamma$$

where $\gamma \approx 0.57721$ (Euler-Mascheroni constant).

Asymptotic Behavior

$$E[T] \sim n \ln n + \gamma n$$

Variance

$$\text{Var}(T) = \frac{\pi^2 n^2}{6} - n H_n \quad (\text{approximately})$$

6. Step-by-Step Algorithm

To compute expected number of draws to collect all n coupons:

1. Initialize expected = 0.
 2. For $k = 0$ to $n-1$ (where k = number of coupons already collected):
 - a. Probability of getting a new coupon = $(n-k)/n$.
 - b. Expected draws to get next new coupon = $n/(n-k)$.
 - c. Add to total.
 3. Return total.
-

7. Dry Run

Problem: Expected boxes to collect all 5 toy types.

Coupons already have (k)	New coupon probability	Expected draws for next	Running total
0	$5/5 = 1$	$5/5 = 1$	1
1	$4/5 = 0.8$	$5/4 = 1.25$	2.25
2	$3/5 = 0.6$	$5/3 \approx 1.67$	3.92
3	$2/5 = 0.4$	$5/2 = 2.5$	6.42
4	$1/5 = 0.2$	$5/1 = 5$	11.42

Answer: $E[T] = 5 \times (1 + 1/2 + 1/3 + 1/4 + 1/5) = 5 \times (2.2833) \approx 11.42$ boxes.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Expected number of draws to collect all n coupons
double couponCollectorExpected(int n) {
    double expected = 0.0;
    for (int k = 1; k <= n; k++) {
        expected += 1.0 / k;
    }
    return n * expected; // n * H_n
}

// Expected using harmonic number approximation for large n
double couponCollectorApprox(int n) {
    const double EULER = 0.5772156649;
    return n * (log(n) + EULER + 0.5 / n); // More accurate approximation
}

// Probability of having collected all n coupons after exactly k draws
// (Using inclusion-exclusion – complex, not recommended for large n)
double probabilityAllCollected(int n, int k) {
    // P(all collected by k draws) = 1 - P(missing at least one)
    // Using inclusion-exclusion would be  $O(n \cdot 2^n)$  – only for small n
    // For practical purposes, we often simulate or use approximation
    vector<double> dp(n + 1, 0.0);
    dp[0] = 1.0; // P(0 coupons collected after 0 draws)

    // DP: dp[i] = P(i distinct coupons collected)
    for (int draw = 1; draw <= k; draw++) {
        vector<double> ndp(n + 1, 0.0);
        for (int i = 0; i < n; i++) {
            if (dp[i] == 0) continue;
            double pNew = (double)(n - i) / n;
            double pOld = (double)i / n;
            ndp[i + 1] += dp[i] * pNew; // Got a new coupon
            ndp[i] += dp[i] * pOld; // Got a duplicate
        }
        dp = ndp;
    }
    return dp[n]; // P(all n collected)
}
```

```

}

// Expected number of distinct coupons after k draws
double expectedDistinct(int n, int k) {
    // Using linearity of expectation
    // P(coupon i is NOT drawn in k draws) = ((n-1)/n)^k
    // P(coupon i IS drawn) = 1 - ((n-1)/n)^k
    // Expected distinct = n * (1 - ((n-1)/n)^k)
    return n * (1.0 - pow((double)(n-1)/n, k));
}

int main() {
    int n = 5;
    cout << "Expected draws to collect all " << n << " coupons: "
         << couponCollectorExpected(n) << "\n"; // ≈ 11.42

    cout << "Approximation for n=100: " << couponCollectorApprox(100) << "\n";
    cout << "Exact for n=100: " << couponCollectorExpected(100) << "\n";

    cout << "Expected distinct after 10 draws (n=10): "
         << expectedDistinct(10, 10) << "\n";

    cout << "P(all 5 collected in 20 draws): "
         << probabilityAllCollected(5, 20) << "\n";

    return 0;
}

```

9. Python Implementation

```

import math

def coupon_collector_expected(n: int) -> float:
    expected = 0.0
    for k in range(1, n + 1):
        expected += 1.0 / k
    return n * expected

def coupon_collector_approx(n: int) -> float:
    EULER = 0.5772156649
    return n * (math.log(n) + EULER + 0.5 / n)

def probability_all_collected(n: int, k: int) -> float:
    """DP for probability of collecting all n coupons in k draws"""
    dp = [0.0] * (n + 1)
    dp[0] = 1.0
    for _ in range(k):
        ndp = [0.0] * (n + 1)
        for i in range(n):
            if dp[i] == 0:
                continue
            p_new = (n - i) / n
            p_old = i / n
            ndp[i + 1] += dp[i] * p_new
            ndp[i] += dp[i] * p_old
        dp = ndp
    return dp[n]

```

```
def expected_distinct(n: int, k: int) -> float:
    return n * (1.0 - ((n - 1) / n) ** k)

n = 5
print(f"Expected draws to collect all {n} coupons: {coupon_collector_expected(n):.2f}")
print(f"Approximation for n=100: {coupon_collector_approx(100):.2f}")
print(f"Expected distinct after 10 draws (n=10): {expected_distinct(10, 10):.2f}")
print(f"P(all 5 collected in 20 draws): {probability_all_collected(5, 20):.4f}")
```

10. Code Explanation

- **couponCollectorExpected**: Sums harmonic series, multiplies by n. Simple $O(n)$ loop.
- **couponCollectorApprox**: Uses $\ln(n) + \gamma$ approximation for large n ($O(1)$).
- **probabilityAllCollected**: DP tracking distribution of number of distinct coupons. $O(k \cdot n)$ time.
- **expectedDistinct**: Uses linearity of expectation. $O(1)$ formula.
- The DP for probability uses the fact that when you have i coupons, the next draw is new with probability $(n-i)/n$.

11. Complexity Analysis

Operation	Time	Space
Expected draws (exact)	$O(n)$	$O(1)$
Expected draws (approx)	$O(1)$	$O(1)$
Probability all collected (DP)	$O(k \cdot n)$	$O(n)$
Expected distinct after k draws	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identification	Approach	Example
Basic coupon collector	"collect all n types"	$E = n \cdot H_n$	Toy boxes
Unequal probabilities	"different probabilities for each type"	Weighted coupon collector	Rare vs common cards
Collecting pairs	"need both items of each type"	Modified coupon collector	Collecting pairs of shoes
Group collecting	"get multiple copies of each"	Negative binomial sum	Pokemon cards
Coupon collector with multiple draws	"draw k coupons at a time"	Modified analysis	Card packs with multiple cards

13. Common Mistakes

- Assuming expected draws = n (only true if draws are without replacement).
- Forgetting the harmonic series and using $n/2$ or similar.
- Not handling large n (harmonic series approximation is important).
- Confusing coupon collector with the birthday problem.
- Using integer division in C++ for harmonic series.
- Assuming the distribution is symmetric (it's heavily right-skewed).

14. Edge Cases

- $n = 1$: expected draws = 1 (always get the only coupon).
- $n = 0$: undefined (no coupons to collect).
- Very large n : use approximation to avoid $O(n)$ loop.
- $k = 0$: expected distinct = 0.
- Unequal probabilities: the basic formula doesn't apply.

15. Variations

Variation	What Changes	When Used	Importance
Weighted coupon collector	Each coupon has different probability	Real-world collections (rare vs common)	High
Multiple copies needed	Need r copies of each	Loyalty card stamps	Moderate
Batch draws	Get m coupons per draw	Card packs	High for CP
Coupon collector with time	Coupons expire	Real-world applications	Low for CP

16. Related Concepts

- **Harmonic Numbers**: The sum $H_n = 1 + 1/2 + \dots + 1/n$ appears constantly.
- **Geometric Distribution**: Each "new coupon" phase is a geometric random variable.
- **Linearity of Expectation**: The key tool for deriving the expected value.
- **Birthday Problem**: The dual — how many draws until you see a duplicate.

17. Practice Problems

Easy

- **Coupon Collector** (GFG) — Basic expected value calculation.
- **Collecting Numbers** (CSES 1654) — Related permutation problem.

Medium

- **Collecting Coins** (Codeforces) — Weighted coupon collector.
- **Random Pick with Weight** (LeetCode 528) — Non-uniform sampling.
- **K-th Smallest in Multiplication Table** (LeetCode 668) — Counting variation.

Hard

- **Coupon Collector with Unequal Probabilities** (Codeforces) — Advanced analysis.
- **Expected Number of Distinct Elements** (Codeforces) — Combined with other techniques.
- **Collecting Numbers** (IOI) — Complex variation.

18. Interview Explanation

"The Coupon Collector problem asks how many random draws you need to collect all n distinct types. The expected number is n times the n th harmonic number, approximately $n \ln(n)$. The key insight is to break the process into phases: from having k coupons to getting the $(k+1)$ th, the probability of success is $(n-k)/n$, so the expected number of draws in that phase is $n/(n-k)$. By linearity of expectation, the total expected draws is the sum $n \cdot H_n$. This is a classic result used in randomized algorithm analysis and probability problems."

19. Revision Notes

- $E[T] = n \times H_n \approx n \cdot \ln(n) + \gamma \cdot n$
- $H_n = 1 + 1/2 + 1/3 + \dots + 1/n$
- Each phase is geometric: $E[\text{draws for next new}] = n/(n-k)$
- Harmonic series approximation: $H_n \approx \ln(n) + 0.577$
- Last coupon takes the longest (expected n draws)
- Linearity of expectation is the proof technique
- Variance is large: $\sim \pi^2 n^2 / 6$

20. Final Cheat Sheet

Formula	Value	Use
$E[T] = n \cdot H_n$	Exact expectation	Small n , exact answer
$E[T] \approx n \cdot \ln(n) + \gamma \cdot n$	Approximation	Large n
$H_n \approx \ln(n) + \gamma$	Harmonic approx	Quick calculation
Last coupon	Expected n draws	Hardest to get
Distinct after k draws	$n \cdot (1 - ((n-1)/n)^k)$	Linearity of expectation

RANDOM WALKS

1. Overview

A random walk is a stochastic process where a particle moves randomly on a state space (usually a graph or integer line). At each step, it moves to a neighboring state according to some probability distribution. Random walks are fundamental in probability, physics, and computer science.

2. Intuition

Simple Explanation

Imagine a drunk person standing on a number line at position 0. Every minute, they take a step left with probability $1/2$ or a step right with probability $1/2$. That's a simple 1D random walk. Where will they be after n steps? On average, at 0 — but the actual distance grows as $\sqrt{n}$.

Analogy: The Drunkard's Walk

The drunkard's walk is the classic analogy. The drunk doesn't remember where they've been, so each step is independent of the past. Over time, they tend to wander further from the starting point, but the expected position is always where they started.

Step-by-Step Reasoning

1. Start at a fixed position (usually 0).
2. At each step, choose a random direction uniformly.
3. Move one unit in that direction.
4. Record the new position.
5. Repeat for n steps.

Why It Works

Random walks model diffusion and Brownian motion. The key properties:

- **Memoryless:** Each step is independent (Markov property).
 - **Symmetric:** Equal probability in each direction.
 - **Expected position:** $E[X_n] = 0$ (for symmetric walk).
 - **Expected absolute distance:** $E[|X_n|] \approx \sqrt{2n/\pi}$.
-

3. When to Use It

- Analyzing particle movement, diffusion, Brownian motion
 - Modeling stock prices (random walk hypothesis)
 - PageRank and web surfing
 - Monte Carlo methods for solving PDEs
 - Problems about "particle moving on a line/grid"
 - Expected hitting time problems
 - Gambler's ruin problems
 - MCMC (Markov Chain Monte Carlo) sampling
-

4. When Not to Use It

- When movement has a bias/drift that you need to model separately
 - When the walker has memory of past positions (use self-avoiding walk)
 - When the state space is infinite and you need to avoid divergence
 - When deterministic movement suffices
 - When the problem requires exact position distribution (use binomial)
-

5. Core Concepts

Simple Random Walk (1D)

Let S_n = position after n steps, where each step is ± 1 with equal probability. $S_n = X_1 + X_2 + \dots + X_n$, where $X_i \in \{-1, +1\}$ with equal probability.

Expected Position

$E[S_n] = 0$ (symmetric walk).

Variance

$\text{Var}(S_n) = n$ (each step has variance 1, independent).

Root Mean Square Distance

$\sqrt{E[S_n^2]} = \sqrt{n}$.

Hitting Time

Expected number of steps to reach a specific position.

Recurrence

In 1D and 2D, simple random walk is **recurrent** (returns to origin with probability 1). In 3D+, it's **transient** (may never return).

Gambler's Ruin

Probability of reaching position a before $-b$, starting from 0: $P(\text{reach } a \text{ before } -b) = b/(a+b)$ for fair game.

6. Step-by-Step Algorithm

To simulate a 1D random walk for n steps:

1. Initialize position = 0.
2. For step = 1 to n: a. Generate a random number r in $[0, 1)$. b. If $r < 0.5$, position += 1 (step right). c. Else, position -= 1 (step left).
3. Return position (or the entire path).

7. Dry Run

Problem: Simulate a 5-step 1D random walk starting at 0.

Step	Random #	Move	Position
0	—	—	0
1	0.23	Right (+1)	1
2	0.81	Left (-1)	0
3	0.15	Right (+1)	1
4	0.67	Left (-1)	0
5	0.92	Left (-1)	-1

Final position after 5 steps: -1.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class RandomWalk1D {
private:
    int position;
    mt19937 rng;

public:
    RandomWalk1D(int start = 0)
        : position(start), rng(chrono::steady_clock::now().time_since_epoch().count()) {}

    // Simulate one step
    int step() {
        double r = uniform_real_distribution<double>(0.0, 1.0)(rng);
        position += (r < 0.5) ? 1 : -1;
        return position;
    }

    // Simulate n steps, return entire path
    vector<int> simulate(int steps) {
        vector<int> path = {position};
        for (int i = 0; i < steps; i++) {
            step();
            path.push_back(position);
        }
        return path;
    }
};
```

```

    }

    // Reset position
    void reset(int start = 0) {
        position = start;
    }

    int getPosition() const { return position; }
};

// Probability of being at position x after n steps
double binomialProbability(int n, int x) {
    // Need (n + x) to be even, and |x| <= n
    if ((n + x) % 2 != 0 || abs(x) > n) return 0.0;
    int k = (n + x) / 2; // number of right steps
    // P = C(n, k) / 2^n
    double logProb = lgamma(n + 1) - lgamma(k + 1) - lgamma(n - k + 1) - n * log(2);
    return exp(logProb);
}

// Gambler's ruin: probability of reaching a before -b starting from 0
double gamblersRuin(int a, int b) {
    // For fair game: P = b / (a + b)
    return (double)b / (a + b);
}

// Expected hitting time to reach position a or -b
double expectedHittingTime(int a, int b) {
    // For fair game: E = a * b
    return (double)a * b;
}

int main() {
    RandomWalk1D rw(0);

    // Simulate 10 steps
    auto path = rw.simulate(10);
    cout << "Path: ";
    for (int p : path) cout << p << " ";
    cout << "\n";
    cout << "Final position: " << rw.getPosition() << "\n";

    // Probability of being at position 2 after 10 steps
    cout << "P(position=2 after 10 steps): " << binomialProbability(10, 2) << "\n";

    // Gambler's ruin: reach +5 before -3
    cout << "P(reach +5 before -3): " << gamblersRuin(5, 3) << "\n";
    cout << "Expected hitting time: " << expectedHittingTime(5, 3) << "\n";

    // Expected absolute distance after n steps (approximation)
    int n = 100;
    double expectedAbsDist = sqrt(2.0 * n / M_PI);
    cout << "Expected |position| after " << n << " steps: ~" << expectedAbsDist << "\n";

    return 0;
}

```

9. Python Implementation

```

import random
import math
from typing import List

class RandomWalk1D:
    def __init__(self, start: int = 0):
        self.position = start

    def step(self) -> int:
        self.position += 1 if random.random() < 0.5 else -1
        return self.position

    def simulate(self, steps: int) -> List[int]:
        path = [self.position]
        for _ in range(steps):
            self.step()
            path.append(self.position)
        return path

    def reset(self, start: int = 0):
        self.position = start

    def get_position(self) -> int:
        return self.position

def binomial_probability(n: int, x: int) -> float:
    if (n + x) % 2 != 0 or abs(x) > n:
        return 0.0
    k = (n + x) // 2
    return math.comb(n, k) / (2 ** n)

def gamblers_ruin(a: int, b: int) -> float:
    return b / (a + b)

def expected_hitting_time(a: int, b: int) -> float:
    return a * b

# Simulation
rw = RandomWalk1D(0)
path = rw.simulate(10)
print(f"Path: {path}")
print(f"Final position: {rw.get_position()}")

# Binomial probability
print(f"P(position=2 after 10 steps): {binomial_probability(10, 2):.6f}")

# Gambler's ruin
print(f"P(reach +5 before -3): {gamblers_ruin(5, 3):.4f}")
print(f"Expected hitting time: {expected_hitting_time(5, 3)}")

# Expected absolute distance
n = 100
expected_abs = math.sqrt(2 * n / math.pi)
print(f"Expected |position| after {n} steps: ~{expected_abs:.2f}")

```

10. Code Explanation

- `RandomWalk1D` class: Simulates 1D random walk. Uses Mersenne Twister for better randomness.

- `step()`: Moves ± 1 with equal probability. Returns new position.
- `simulate(steps)`: Returns full path including start position.
- `binomialProbability`: $P(\text{position} = x \text{ after } n \text{ steps}) = C(n, (n+x)/2) / 2^n$. Uses log-gamma for numerical stability.
- `gamblersRuin`: For fair game, probability of reaching a before $-b$ is $b/(a+b)$.
- `expectedHittingTime`: For fair game starting at 0, expected steps to hit a or $-b$ is $a \cdot b$.

11. Complexity Analysis

Operation	Time	Space
Single step	$O(1)$	$O(1)$
Simulate n steps	$O(n)$	$O(n)$ if path stored, $O(1)$ if not
Binomial probability	$O(1)$	$O(1)$
Gambler's ruin	$O(1)$	$O(1)$
Expected hitting time	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identification	Approach	Example
Symmetric random walk	"step left/right with equal probability"	Binomial distribution	Drunkard's walk
Biased random walk	"probability p of right, q of left"	Martingale, gambler's ruin	Stock with drift
Gambler's ruin	"probability of reaching A before B "	Solve recurrence	Casino game
2D random walk	"move on a grid"	Markov chain on grid	Particle diffusion
Random walk on a graph	"move to a random neighbor"	Markov chain theory	PageRank
Self-avoiding walk	"cannot revisit nodes"	Polymers, protein folding	Physics simulation

13. Common Mistakes

- Assuming $E[|S_n|] = \sqrt{n}$ (it's actually $\sqrt{(2n/\pi)}$).
- Forgetting that 1D random walk is recurrent (returns to origin infinitely often).
- Confusing hitting time with return time.
- Not handling the parity condition: after n steps, position parity = $n \bmod 2$.
- Using symmetric random walk when the problem has a bias.
- Assuming 3D random walk is recurrent (it's transient).

14. Edge Cases

- $n = 0$ (position = start, always).
- $n = 1$ (position = ± 1 with equal probability).
- Very large n (use normal approximation).
- Start position $\neq 0$ (shift all results).
- Absorbing barriers (gambler's ruin with boundaries).
- Reflecting barriers (bounce back at boundaries).

15. Variations

Variation	What Changes	When Used	Importance
Biased random walk	$p \neq 0.5$	Drift models, stock prices	High
2D random walk	Move on grid	Diffusion, image processing	High
Random walk on a graph	General graph	PageRank, MCMC	High
Self-avoiding walk	No revisiting	Polymers, biology	Moderate
Lazy random walk	$P(\text{stay}) > 0$	Mixing time analysis	Moderate
Continuous-time random walk	Continuous time	Physics, finance	Low for CP

16. Related Concepts

- **Markov Chains:** Random walks are a special case of Markov chains.
- **Brownian Motion:** Continuous limit of random walk.
- **Diffusion:** Physical analogue of random walks.
- **Martingales:** Expected future value equals current value.
- **Harmonic Functions:** Functions satisfying averaging property over neighbors.

17. Practice Problems

Easy

- **Random Walk** (GFG) — Basic simulation.
- **Robot Return to Origin** (LeetCode 657) — Simple walk check.

Medium

- **Random Walk on a Grid** (CSES) — 2D random walk.
- **Gambler's Ruin** (Codeforces) — Probability of hitting boundaries.
- **Where Will the Ball Fall** (LeetCode 1706) — Grid random walk.

Hard

- **Random Walk Expected Time to Reach Goal** (Codeforces) — System of equations.
- **Probability of a Random Walk** (CSES) — Complex probabilistic analysis.
- **PageRank** (LeetCode) — Random walk on web graph.

18. Interview Explanation

"A random walk is a process where a particle moves randomly at each step. In 1D, the particle moves left or right with equal probability. The expected position after n steps is 0, but the expected absolute distance grows as $\sqrt{n}$. The walk is recurrent in 1D and 2D — it returns to the origin infinitely often — but transient in 3D. The gambler's ruin problem analyzes the probability of hitting one boundary before another. Random walks are the foundation of diffusion models, stock price theory, and PageRank."

19. Revision Notes

- 1D symmetric: $S_n = \sum X_i$, $X_i \in \{\pm 1\}$
- $E[S_n] = 0$, $\text{Var}(S_n) = n$, $\text{RMS} = \sqrt{n}$
- $P(\text{position} = x \text{ after } n \text{ steps}) = C(n, (n+x)/2) / 2^n$
- 1D and 2D: recurrent; 3D+: transient
- Gambler's ruin: $P(\text{reach } a \text{ before } -b) = b/(a+b)$

- Hitting time (fair): $E = a \cdot b$
- Expected absolute distance: $\sim \sqrt{2n/\pi}$

20. Final Cheat Sheet

Property	Formula	Notes
Expected position	$E[S_n] = 0$	Symmetric walk
Variance	$\text{Var}(S_n) = n$	Each step adds 1 variance
RMS distance	$\sqrt{n}$	Standard deviation
Position distribution	$\text{Binomial}/2^n$	$C(n, (n+x)/2) / 2^n$
Gambler's ruin	$b/(a+b)$	Reach a before $-b$
Hitting time	$a \cdot b$	Expected steps to hit boundary
Recurrence	1D, 2D: recurrent	Returns with prob 1
	3D+: transient	May never return
Expected	S_n	

Pro Tip: These probability and random process topics are heavily tested in SDE placements at companies like Google, Microsoft, Amazon, and in competitive programming (Codeforces, CodeChef, AtCoder). The key skills are: (1) breaking problems using indicator variables and linearity of expectation, (2) recognizing when a process is a Markov chain or random walk, (3) computing expected values via geometric series, and (4) using Bayes theorem for conditional probability problems. Practice with the listed problems to build intuition.